# LUnit

## Astemes - Anton Sundqvist

*Anton Sundqvist*

*Copyright © 2025 - 2026 Astemes*

# Table of contents


1. LUnit Basics 4
   - 1.1 Prerequisites 4
   - 1.2 Creating a Test Case 4
   - 1.3 Adding a Test Method 4
   - 1.4 Using Assertions 6
   - 1.5 Adding utility VIs 6
   - 1.6 Using the Setup and Teardown methods 7
   - 1.7 Running the Test Case 7
   - 1.8 Organizing Tests 0
2. Framework Architecture 0
   - 2.1 General Architecture 0
   - 2.2 Test Case 0
   - 2.3 Test Methods 0
   - 2.4 Assertions 0
   - 2.5 Test Runner 0
   - 2.6 Test Finder 0
   - 2.7 LabVIEW API 0
   - 2.8 Command Line Interface 0
3. Profiling Tools 0
   - 3.1 Execution Profiler 0
   - 3.2 Code Coverage Analyzer 0
4. CI Integration 0
   - 4.1 Executing Tests from the Command Line 0
   - 4.2 Capturing the Test Results 0
   - 4.3 GitHub Actions Example 0
   - 4.4 Jenkins Example 0
5. Real-Time Systems 0
   - 5.1 Testing with hardware 0
   - 5.2 About running tests on a Real-Time target 0
   - 5.3 Working with tests in LabVIEW Real-Time 0
   - 5.4 Running LUnit on a Real-Time target 0
6. Writing Unit Tests 0
   - 6.1 Why bother writing unit tests? 0
   - 6.2 How to write unit tests 0
   - 6.3 How not to write tests 0




Copyright © 2025 - 2026 Astemes

6.4 When to write unit tests 0

6.5 Testing in isolation 0

7. Creating Report Plugins 0

7.1 Getting started 0

7.2 Implementing the Reporting Plugin Interface 0

7.3 Plugin Configuration 0

7.4 Deploying your plugin 0

7.5 Upgrading report plugins from LUnit version 1.x to version 2.x 0

8. License 0

8.1 Astemes LUnit 0

8.2 JKI Flat UI Controls 0



Copyright © 2025 - 2026 Astemes

# 1. LUnit Basics

This document walks through the basic workflow using LUnit to test LabVIEW code. If you prefer to watch a video, an introduction is available here.

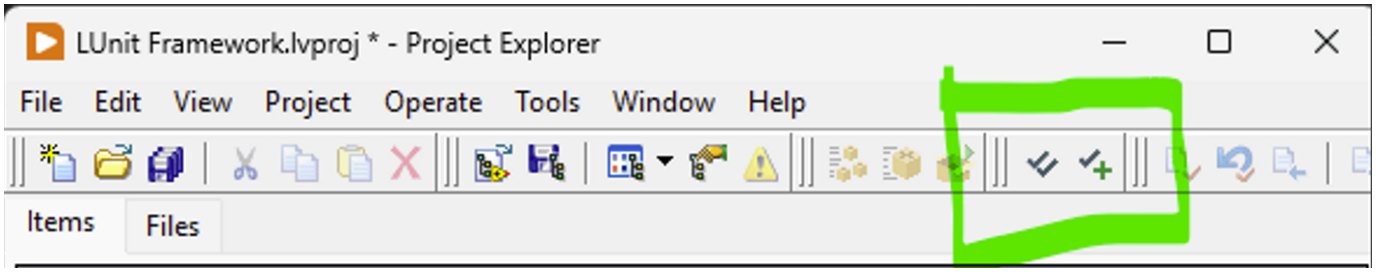
## 1.1 Prerequisites

To follow along with the instructions on this page, you will need to have LabVIEW version 2020 or later installed as well as the LUnit unit testing framework.

## 1.2 Creating a Test Case

All tests you write will belong to a test case class. This is implemented as a LabVIEW class, but you do not need to know anything about object-oriented programming to use it.

To get started, create an empty project and add a test case class to it by clicking the New Test Case button in the toolbar of the LabVIEW project.




You can also do the same from the `Tools > LUnit > New Test Case...` menu option.




Save the test case in a convenient location. Some like to keep the tests next to the code they are testing, and others keep them in a separate folder called `Tests` or similar. A separate top-level directory is often a sensible default option. Keep in mind that tests should not be included in builds and there should be no dependencies pointing from your code to the test code.

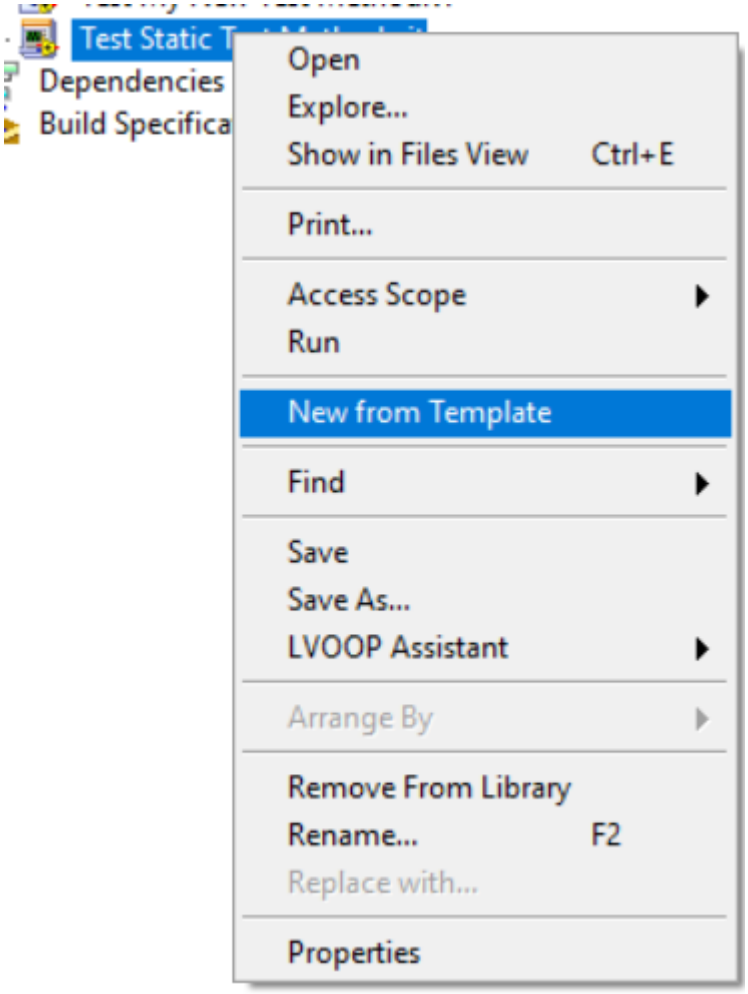
## 1.3 Adding a Test Method

Now you have a test case and may add some test methods to the test case. A test method is a VI belonging to the test case class and will get executed by the framework.

If you are using LUnit version 1.x, the name of the VI **must** start with the four letters "test" (case insensitive). In LUnit 2.0, this requirement has been relaxed and any public VI in a test case class is considered a test and enumerated in the UI. If the VI contains assertions, it will report results when the test is run. Helper VIs should be made `private` within the test case class, which will avoid them being shown and run by the UI. It is **not** recommended to make test VIs *dynamic dispatch*. To create a new test method, right-click on the *Test Method Template.vit* and select `New from Template`.



Copyright © 2025 - 2026 Astemes

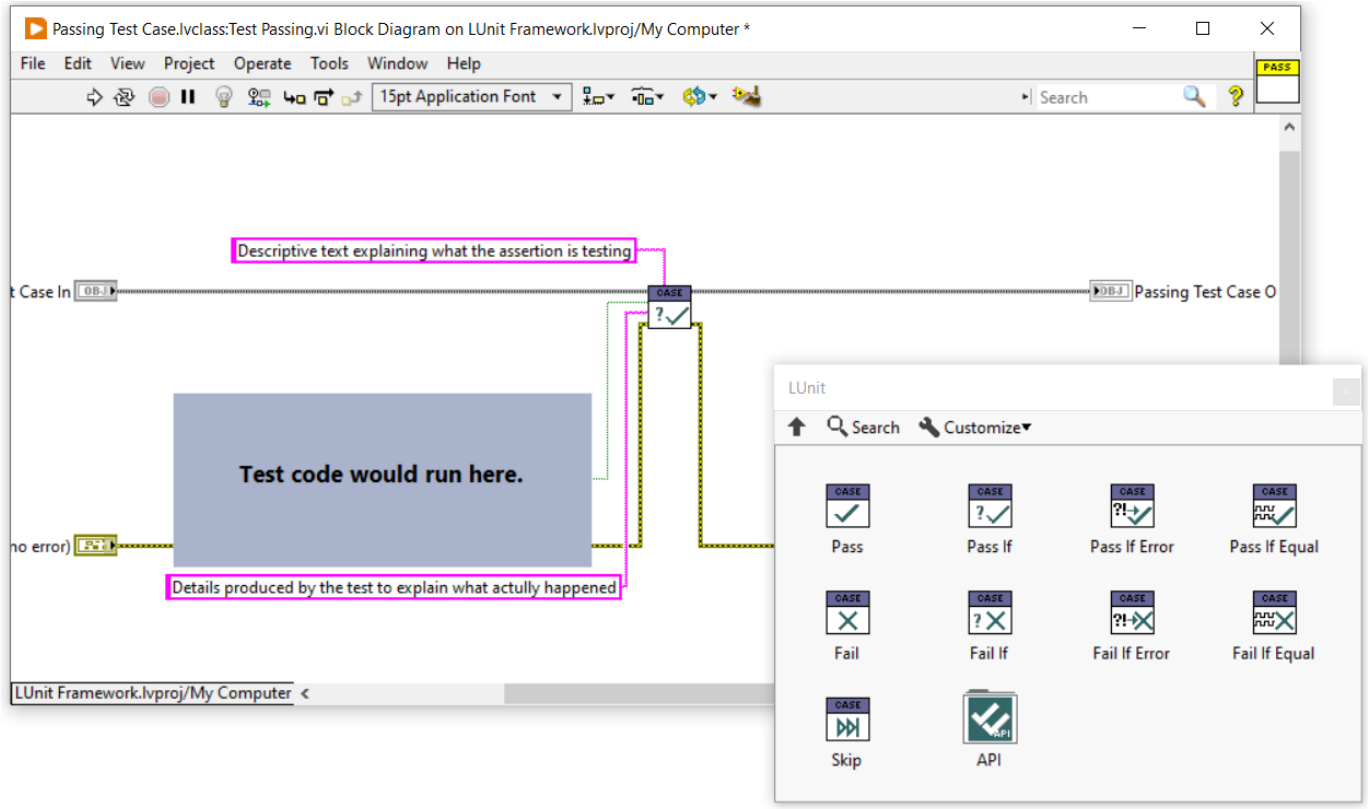



You can create test methods any way you like and you are free to delete the template method. It is important however that the connector pane uses the same pattern of terminals as the template. The error in terminal is optional and never used when running tests by the framework.

You should now make your test method test something useful by implementing the block diagram of the VI. To perform tests you will use the assertions available in the provided palette, or via Quick Drop.



Copyright © 2025 - 2026 Astemes




## 1.4 Using Assertions

The result of each test is determined using assertions. There is a set of assertions to choose from, as shown in the figure above, and the names should be self-explanatory. One test method may contain multiple assertions and the result from each assertion will show up in the result view.

Note that the `Pass if Equal.vi` assertion also works well for array data types, clusters and data classes. The result of comparing arrays will show up in the result view as shown below.




Please note that the `Pass if Equal.vi` assertion will fail if either the type or the value does not match.

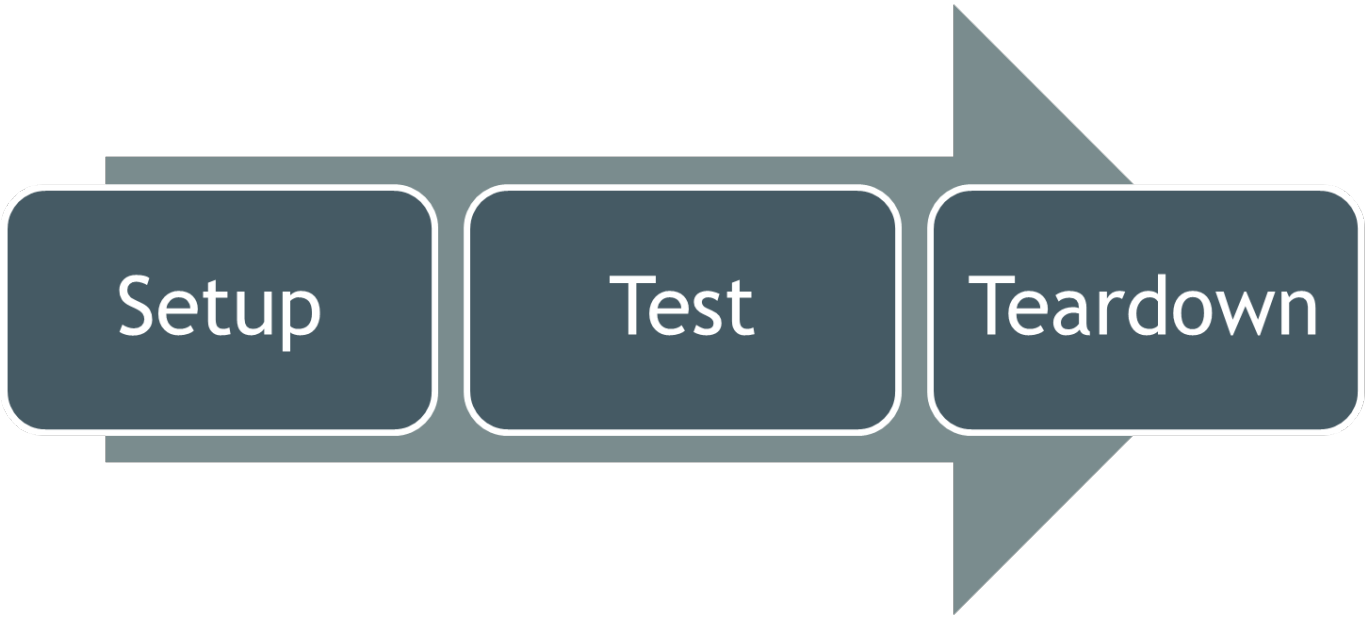
## 1.5 Adding utility VIs

It is common that code is reused between tests belonging to a test case class, and could then be placed in subVIs or so-called test utility VIs. If you create such VIs belonging to the class, make sure to restrict the access scope (*i.e.* make the VIs `private` or `protected`), as the UI enumerates all public VIs in the class.


Copyright © 2025 - 2026 Astemes

## 1.6 Using the Setup and Teardown methods

You can add a Setup and a Teardown method to the test case by overriding the corresponding dynamic dispatch VIs. The Setup VI will run once before each test method in the test case and the Teardown VI will run once after the test method is completed. This is useful in some cases, but should not be overused as it makes the test methods less self-contained. If you need to pass data from the Setup VI to the test method VI or Teardown VI, you can bundle the data into the test case class wire.




## 1.7 Running the Test Case

### 1.7.1 Run Arrow

You can run a single test VI (using the Run Arrow) and it will run and show the user interface with the results of the test. When run this way, the VI is actually run twice. The first run launches the UI, and the second is the actual test execution. In this way, the UI can run the `Setup.vi` dynamic dispatch, the test method and then finally the `Teardown.vi` method even when the test is not started from the UI.

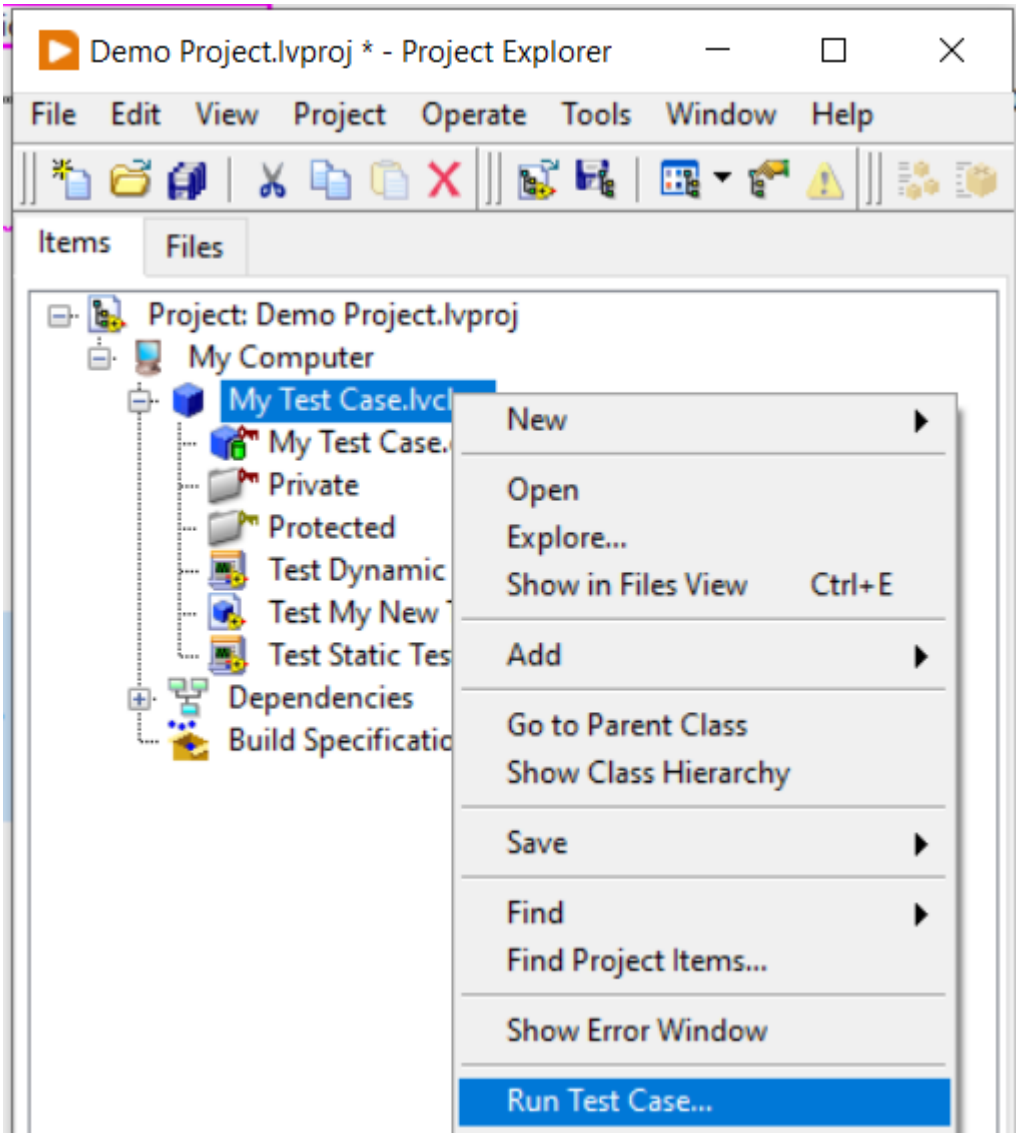
### 1.7.2 Quick Drop

Starting from version 1.10 of LUnit, a Quick Drop plugin is available which allows running tests using a Quick Drop shortcut. From Quick Drop press `Ctrl + L` to run all tests within the current project. If you open Quick Drop from a VI and press `Ctrl + Shift + L` LUnit will run all tests from Test Case classes calling this specific VI. This can be very useful for checking if an edit to a VI caused any test to fail. Please note that this feature requires all tests to be loaded into memory, *i.e.* included in the active project, and it can only check for static links. The last limitation means that the feature will not work for dynamic dispatch VIs, as their call sites are not statically known.

### 1.7.3 Project Window and LUnit UI

To run all tests contained in a test case, you can right click it in the project window and select the `Run Test Case...` menu option.

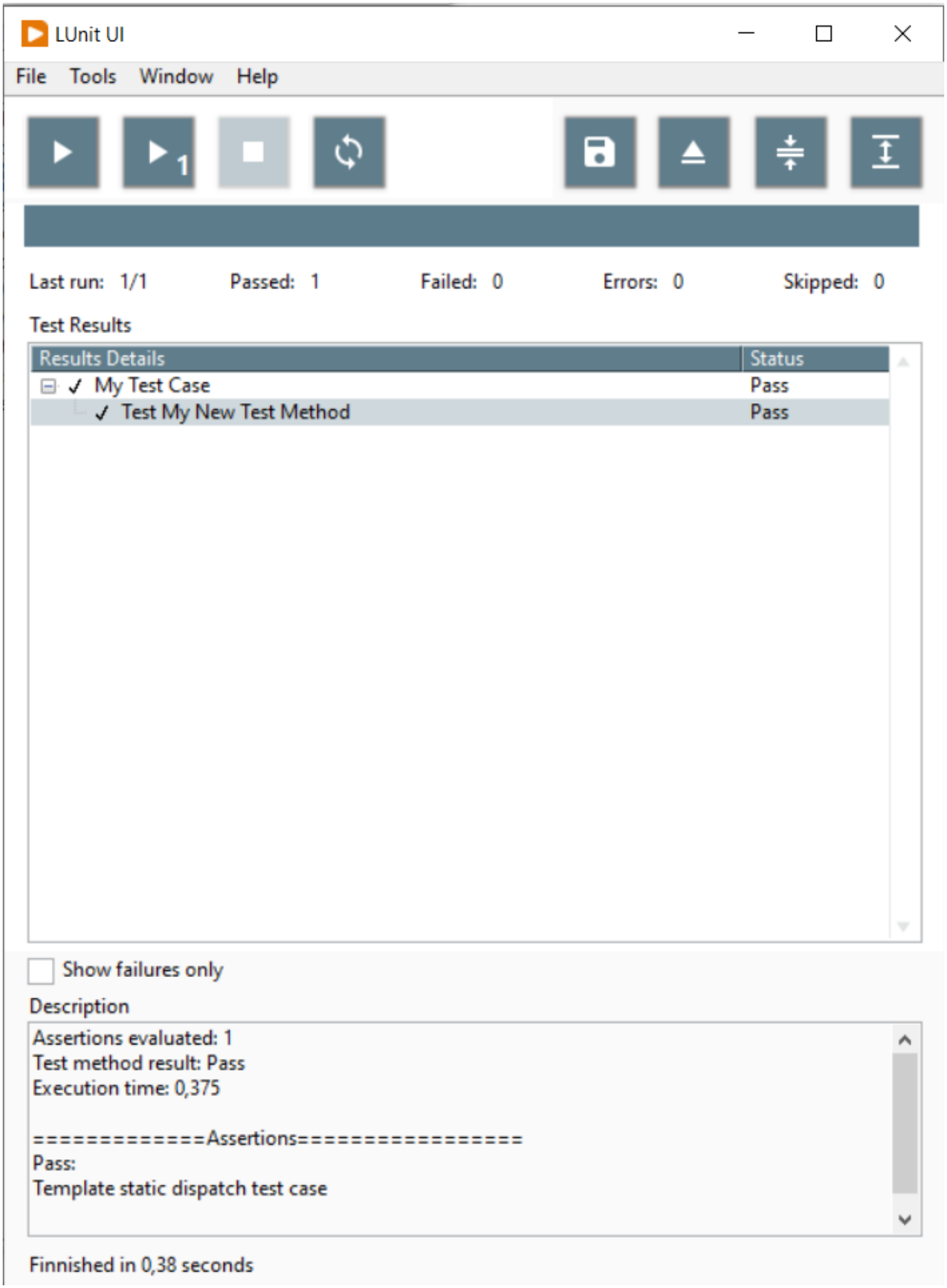

Copyright © 2025 - 2026 Astemes




This will open the test execution user interface and run the test case. You can also launch the user interface from the tools menu through the `Tools > LUnit > LUnit UI...` menu option. This will open the user interface and show all tests in the current project. As the test is run, the results are also shown as visual icon overlays in the project explorer.



Copyright © 2025 - 2026 Astemes






Copyright © 2025 - 2026 Astemes